

READING SHEET

Next.js — Intro & File-Based Routing

Module 3 · Frontend: React & Next.js · Day 36 · Week 8

PROGRAM CodeOps · Full Stack	WORK TYPE Revision + mini-project	EST. TIME 2–3 hours
--	---	-------------------------------

Tip How to use this sheet

This is your take-away for Day 36, the first day of Next.js. Nothing you learned about React stops being true — components, props, state, hooks and effects are all unchanged. What changes is everything around them: routing comes from the folder structure, rendering starts on the server, and the loading and error boundaries you wrote by hand on Day 34 become files with reserved names. Build the route tree as you read.

Part 1 · Why Next.js

React is a library for building user interfaces and nothing more. Every decision beyond that — the build tool, the router, how data is fetched, how the bundle is split — you have been making yourself. Next.js is a **framework**: you put files in the right places and it does the wiring.

YOU CHOSE AND WIRED	NEXT.JS PROVIDES
Vite as the build tool	The build, already configured
React Router for navigation	Routing from the folder structure
A useFetch hook	Data fetching in the component itself
Manual code splitting	Splitting per route, automatically
Plain img tags	An Image component that optimises
Nothing for SEO	A metadata API and server rendering

The limit of client-only rendering

```

// what a search engine receives from a Vite app
<body>
  <div id="root"></div>      <!-- empty -->
  <script src="/assets/index-a3f8.js"></script>
</body>

// the menu only exists after the JavaScript runs

```

The browser downloads the HTML, then the JavaScript, then runs it, then fetches the data, then finally renders. On a slow connection that is several seconds of blank screen — and a crawler that reads the HTML sees nothing at all. A restaurant that cannot be found on Google loses customers, and so does a first screen that takes four seconds.

SERVER RENDERING GAINS	WHY IT MATTERS
HTML arrives complete	Content is visible before JavaScript loads

SERVER RENDERING GAINS	WHY IT MATTERS
Search engines read it	The menu is indexable and shareable
Less JavaScript shipped	Server components send no code to the browser
Data fetched server-side	No waterfall of requests from the phone
Secrets stay on the server	API keys never reach the client

Tip When not to use Next.js

An internal dashboard behind a login gains little from server rendering, and plain React is simpler to reason about. So does a widget embedded in someone else's page. Server rendering also needs somewhere to run — a host with no Node runtime can serve an exported static site but not the dynamic parts. The rule of thumb: public content where SEO matters points to Next.js; a private tool where every screen is interaction points to React with Vite. Knowing when not to use it is part of knowing it.

Part 2 · Setup & Structure

```
terminal - scaffold and run
npx create-next-app@latest addis-eats

TypeScript?      No      ESLint?      Yes
Tailwind CSS?    Yes     App Router?  Yes

cd addis-eats
npm run dev      # http://localhost:3000
```

Say yes to the **App Router**. Next.js has two routing systems: the older pages router still exists in many codebases, but everything new is built on the app router and that is what this module teaches. Accepting Tailwind means the utility-first styling from Module 2 carries straight across.

The folders that matter

```
addis-eats/ - what each part does
app/
  layout.js  every route lives here
  page.js    the shell wrapping all pages
  the "/" route
public/      static files served as-is
  logo.png   -> /logo.png
next.config.mjs framework configuration
```

app/ is the entire router — there is no equivalent of the App.jsx route table you wrote on Day 31. public/ is literal: a file at public/logo.png is served at /logo.png with no import and no build step.

The reserved file names

FILE	WHAT IT CREATES
page.js	A publicly reachable route
layout.js	A shell wrapping this segment and everything below
loading.js	Shown while the segment loads

FILE	WHAT IT CREATES
error.js	Shown when the segment throws
not-found.js	Shown for a missing resource
route.js	An API endpoint instead of a page

These six names are the API. Anything else in `app/` — a component, a helper, a stylesheet — is just a file and creates no route. Only `page.js` makes a folder reachable, which is how you keep components beside the route that uses them without accidentally publishing them.

Reading the build output

```
npm run build

Route (app)      Size    First Load JS
┌   ○ /          1.2 kB    89 kB
├   ○ /menu      2.4 kB    92 kB
└   λ /menu/[id]  1.8 kB    91 kB

// ○ static    λ rendered on request
```

Warning Development is not production

The build output lists every route, how it will be rendered and what it costs in JavaScript — the Day 34 bundle conversation, printed for you on every build. But the dev server compiles routes on demand and is far slower than the real thing: never judge performance, or a rendering strategy, from `npm run dev`.

Part 3 · File-Based Routing

A folder is a URL segment and a `page.js` inside it is the screen. The route table you wrote by hand becomes a folder tree, which means the routes cannot get out of step with the files.

```
app/ - the Addis Eats routes

app/
  page.js      -> /
  menu/page.js -> /menu
  cart/page.js -> /cart
  checkout/page.js -> /checkout
  orders/history/page.js -> /orders/history
```

A page is a component

```
app/menu/page.js

export default function MenuPage() {
  return <main><DishList dishes={dishes} /></main>;
}
```

A page is an ordinary React component with a **default export**. The file name decides the route; the function name is yours. A named export will not be found and the route will fail to build — the single most common mistake in the first hour with Next.js.

Navigating

next/link and next/navigation

```
import Link from "next/link";
<Link href="/menu">Menu</Link>

"use client";
import { useRouter } from "next/navigation";
const router = useRouter();
router.push("/checkout");
```

Link replaces the anchor exactly as it did on Day 31, and useRouter replaces useNavigate. Note the prop is href, not to. Import from next/navigation — never the old next/router — when you are on the app router. Next.js also prefetches linked routes in the background, so navigation feels instant.

Colocating components

app/menu/ — the feature folder, now routable

app/menu/	
page.js	-> /menu
DishList.jsx	not a route — just a component
CategoryBar.jsx	not a route
dishes.js	helper, no route

The feature-grouped folders you planned on Day 35 map directly onto the app directory. Anything used by more than one route belongs *outside* app/ — a components/ or lib/ folder at the project root keeps the routing tree honest.

Part 4 · Dynamic & Special Routes

Square brackets in a folder name make a segment dynamic. Unlike Day 31, you do not call a hook to read it — the framework passes params in as a prop, which is what allows the page to render on the server where hooks are not available.

app/menu/[id]/page.js

```
export default function DishPage({ params }) {
  const { id } = params; // "kitfo"
  return <h1>{id}</h1>;
}
// [...slug] catches /menu/a/b/c as an array
```

Every parameter arrives as a string, exactly as before — convert it before comparing against a numeric id.

Loading and error, by file

app/menu/ — three files, three states

app/menu/	
page.js	the menu itself
loading.js	shown while page.js loads
error.js	shown if page.js throws

```
export default function Loading() {
  return <MenuSkeleton />;
}
```

Warning error.js must be a client component

Next.js wraps the segment in a Suspense boundary and an error boundary for you — the two patterns you wrote by hand on Day 34 become two files. But error.js needs state and an event handler to offer a retry, so it must begin with "use client". Day 38 explains exactly why.

not-found and route groups

```

import { notFound } from "next/navigation";

export default async function DishPage({ params }) {
  const dish = await getDish(params.id);
  if (!dish) notFound(); // renders not-found.js
  return <DishCard {...dish} />;
}

```

a dish that does not exist

Two kinds of missing, as on Day 31. A path nobody claimed renders the root not-found.js automatically. A valid path whose data does not exist is your decision — call notFound() and the framework renders the same screen. Separately, a folder in round brackets such as app/(shop)/menu/page.js groups files without adding a segment: the route is still /menu.

The same routes, both ways

DAY 31 · REACT ROUTER	DAY 36 · NEXT.JS
<Route path="/" ...>	app/page.js
<Route path="menu" ...>	app/menu/page.js
<Route path="menu/:id" ...>	app/menu/[id]/page.js
<Route path="*" ...>	app/not-found.js
<Suspense fallback>	app/menu/loading.js
<ErrorBoundary>	app/menu/error.js

Assigned Reading

Required

1. [nextjs.org — Getting started](#) What / why: installation, project structure and your first route, in the right order.
2. [nextjs.org — Defining routes](#) What / why: folders, page.js, nesting and the full set of routing conventions.

Optional — go deeper

- [Dynamic routes](#) — bracketed segments, catch-all routes and reading params.
- [Loading UI and streaming](#) — how loading.js maps onto Suspense, and what streaming adds.

Exercises

Work these in order and commit as you go.

1. Scaffold a Next.js project with create-next-app, choosing the App Router.
2. Create the home, menu, cart and checkout routes as folders with a page.js in each.

3. Add `app/menu/[id]/page.js` and render the id from params on the screen.
4. Move `DishList` and `CategoryBar` into `app/menu/` and confirm neither creates a route.
5. Link every screen with `next/link`, and add a client component that pushes with `useRouter`.
6. Add `loading.js` and `error.js` to the menu segment, and force each one to appear.
7. Add `not-found.js` and call `notFound()` for a dish id that does not exist.

Mini-Project · Addis Eats on Next.js

Tip What you build

A new Next.js project carrying the whole Addis Eats route tree, built entirely from folders: home, menu, a dynamic dish page, cart and checkout, with loading, error and not-found states declared by file. The components move across unchanged — the point of the exercise is that the routing disappears into the file system.

Requirements

- Five routes created as folders, each with a default-export `page.js`.
- A dynamic `/menu/[id]` route reading params from its props rather than a hook.
- Menu components colocated in `app/menu/` and demonstrably not routable.
- `loading.js` and `error.js` on the menu segment, both proven to render.
- `not-found.js`, reached both by a bad URL and by calling `notFound()`.
- Every navigation using `next/link`, with no plain anchors between internal routes.

A worked start

```
starter – the folder tree to create

app/
  layout.js          root shell (minimal for now)
  page.js            /
  menu/
    page.js          /menu
    loading.js
    error.js         "use client"
    [id]/page.js     /menu/[id]
  cart/page.js       checkout/page.js
  not-found.js
```

What to submit

Your GitHub repository with the full `app/` tree, plus a README listing each route and which file produces it.

Check yourself

- Does `npm run build` list every route you expected, and no routes you did not?
- Is `DishList.jsx` unreachable as a URL even though it sits inside `app/`?
- Does the dish page work when typed directly into the address bar?
- Does throttling the connection reveal your `loading.js`?
- Does a deliberate throw show `error.js`, and an unknown dish id show `not-found.js`?